

OpenMCPSpec: A Specification Framework for Robust, Governed, and Lifecycle-Managed Machine Communicable Processes in LLM-Agent Systems

Pinnamaraju Chaitanya
QEA, Cognizant Technologies
Cognizant Technology Solutions
Hyderabad, India
pinnamaraju.chaitanya@gmail.com

Abstract— The proliferation of Large Language Models (LLMs) driving autonomous agents has introduced significant software engineering challenges to enterprise automation, particularly concerning the fragility, governance, and long-term maintainability of the services (tools) they invoke [3]. Current standards for Model Context Protocols (MCP) are often basic, lacking the necessary context, intents that could be enumerated, discoverability, and seamless integration features for agents from the MCP server provider, thus neglecting the critical non-functional requirements essential for production-grade agent services [6]. This paper introduces the OpenMCPSpec Framework, a novel, extensible specification that formally embeds governance metadata, service lifecycle policies, and LLM reliability hints directly into the service contract [7]. We demonstrate the specification's structure through a formal schema and a Spring Boot-based reference implementation [12]. We propose validation metrics—Argument Correctness Improvement (ACI) and Compliance Pre-emption Rate (CPR)—to quantify the framework's effectiveness in enhancing LLM tool-calling reliability and maintaining enterprise compliance [19].

Keywords— *LLM Agents, Tool Calling, Agentic AI, Artificial Intelligence, Specification, Governance, Lifecycle Management, Model Context Protocol (MCP), Software Engineering.*

I. INTRODUCTION

A. Contextualizing the Shift to Autonomous LLM Agents

The integration of Large Language Models (LLMs) into autonomous software agents signals a major transition in enterprise automation, moving from scripted workflows to dynamic, goal-oriented processes [9]. To execute real-world tasks—ranging from database lookups and content retrieval to complex transactional processing—these agents must interact reliably with external software components, which this paper refers to as Machine Communicable Processes (MCPs) [12].

This agentic paradigm shift requires a rethinking of how external services are defined and consumed [1]. LLMs possess inherent limitations, notably their knowledge cutoff [11] (which fixes their internal knowledge at a specific training date) and their inability to directly access real-time, private, or complex operational systems. Robust tool-calling mechanisms are essential to bridge this gap, often utilizing approaches derived from Retrieval-Augmented Generation (RAG) [10] paradigms or direct, secure API invocation to provide dynamic,

relevant context. The success of an autonomous agent hinges entirely on its ability to accurately select the correct tool, map the natural language intent to rigid parameters, and execute the service reliably [5].

B. The Deficiencies of Current Tool Specification Standards

The Existing standards like OpenAPI excels at documentation and client SDK generation for human-centric applications, but lacks native support for conversational elements crucial for agent-model interaction [7][14], such as LLM prompt augmentation or explicit data sensitivity flagging.

The Model Context Protocol (MCP) emerged to specifically address the needs of AI agents, providing a capability-centric focus where the server self-describes the tools it offers. However, basic MCPs typically offer only simplified function definitions (name, description, parameters). While functional, they remain critically deficient [5][19] in addressing essential Non-Functional Requirements (NFRs) and contract or specification driven strategy required for enterprise deployment:

1. **Fragility in Tool-Calling:** LLMs struggle to translate nuanced natural language intents into rigidly defined tool parameters without sufficient external context, resulting in unreliable tool execution.
2. **Governance Debt:** Enterprise requirements for security (Role-Based Access Control, RBAC), data sensitivity (PII/GDPR), and auditing are generally implemented outside the service contract. This decoupling leads to "governance debt," where security policy is implemented reactively in middleware or custom agent logic, resulting in compliance gaps and difficult, inconsistent maintenance across the agent mesh.
3. **Lifecycle Management:** Service evolution, continuous deployment (AgentOps), and version updates frequently lead to brittle integrations because agents lack a standardized, runtime mechanism for discovering version compatibility.

C. The OpenMCPSpec Solution and Key Contributions

The OpenMCPSpec framework is proposed as a comprehensive schema standard designed to rectify these shortcomings by integrating NFRs directly into the service

contract, thereby elevating Machine Communicable Processes (MCPs) into governed software artifacts. The underlying architectural necessity driving this integration is the observation that for agent systems to achieve enterprise maturity, controls related to Governance, Reliability, and Lifecycle must be enforced at the interface definition layer, rather than relying on external infrastructure layers. This radical shift ensures policy and hints are mandated at the protocol level, guaranteeing consistency and proactive compliance before runtime execution.

The key contributions of OpenMCPSpec framework are:

1. **A Formalized OpenMCPSpec Schema:** A specification that mandates governance (pii, roles) and lifecycle metadata, ensuring the contract serves as the single source of truth for both functionality and policy.
2. **Declarative Reliability:** Integration of `nlp_hints` to formalize prompt engineering by providing the agent's orchestration layer with domain-specific vocabulary, directly enhancing LLM tool-calling accuracy.
3. **Contract-Enforced Governance:** Embedding critical governance metadata to enable **Compliance Pre-emption**—the ability to perform security checks or data sanitization at the binding layer *before* business logic is invoked.
4. **Dynamic Lifecycle Management:** Requiring a mandatory enumeration section with a versioning object (compatibility field) to facilitate machine-readable runtime discovery and version negotiation, minimizing brittleness during service updates.
5. **Validation Metrics:** Definition of **Argument Correctness Improvement (ACI)** and **Compliance Pre-emption Rate (CPR)** to empirically measure the benefits derived from the reliability hints and governance features, respectively.

II. RELATED WORK AND THE AGENT PROTOCOL LANDSCAPE

A. Foundational Work in LLM Reasoning and Tool Utilization

The development of OpenMCPSpec is anchored in foundational research showing that LLMs can overcome inherent limitations by leveraging external tools [3]. LLMs inherently excel at rapid, intuitive decision-making (akin to System 1 thinking) but often lack the depth for complex, multi-step logical processes (System 2 reasoning).

Early work, such as **Toolformer** [2], demonstrated that language models could be trained in a self-supervised manner to decide when to call external APIs (e.g., calculators or search engines), what arguments to pass, and how to incorporate the results. Building on this, the **ReAct** [4] (Reasoning and Acting) framework synergized internal reasoning traces with external actions, significantly improving the LLM's capability to perform complex tasks with higher success rates and better efficiency, often requiring only a handful of few-shot prompts for guidance.

Despite these advances, a significant vulnerability persists: tool-calling fragility. LLMs, even with advanced reasoning frameworks, are susceptible to errors when translating highly nuanced user requests—expressed in natural language—into the rigid constraints of API parameters. External factors like prompt injection attacks, which can trick the LLM into specifying internal API keys or triggering unauthorized requests, further demonstrate the need for robust defence mechanisms beyond the model's internal safety alignment. The integration of structured protocols that provide explicit, domain-specific context is necessary to counteract this systemic fragility.

B. The Emerging Model Context Protocol (MCP) Ecosystem

The Model Context Protocol (MCP) has rapidly become a focus area in AI research, aiming to standardize the context exchange between AI clients (Hosts) and external tool servers. Recent surveys published in 2025 confirm this accelerated push toward standardization [1][16], exploring the architecture, state-of-the-art implementations, and future challenges of MCPs.

A crucial challenge in the growing agentic landscape is interoperability. Researchers are actively surveying various agent communication standards, including MCP, the Agent Communication Protocol (ACP), the Agent-to-Agent Protocol (A2A), and the Agent Network Protocol (ANP) [13][15][17]. While these protocols define how agents communicate with each other, MCP focuses specifically on how agents interact with external services.

A key scalability concern for the entire ecosystem is the overhead of tool discovery. Providing an LLM with full tool definitions for hundreds or thousands of services consumes excessive tokens, dramatically increasing latency and cost, a recognized problem in high-volume deployments. This concern has spurred the development of solutions like **MCP-Zero**, which focuses on active tool discovery to efficiently select tools from large candidate pools. OpenMCPSpec addresses this architectural challenge by making the enumeration section mandatory, ensuring the specification structurally supports efficient [12][6], token-saving tool discovery methods from the outset, rather than relying on the LLM to process a massive initial context window.

C. Protocol Comparative Analysis: Defining the OpenMCPSpec Gap

The problem addressed by OpenMCPSpec is not merely descriptive accuracy but achieving *production readiness* and compliance. Traditional software engineering places governance and lifecycle management in external infrastructure layers (API gateways, monitoring systems). OpenMCPSpec integrates these non-functional concerns directly into the protocol definition, enforcing policy and reliability constraints at the service interface itself.

Table I: Comparative Feature Analysis: OpenAPI vs. Basic MCP vs. OpenMCPSpec

TABLE I.

Comparative Feature Analysis: OpenAPI vs. Basic MCP vs. OpenMCPSpec

Feature	OpenAPI (REST/Traditional)	Basic Model Context Protocol (MCP)	OpenMCPSpec Framework
Primary Goal	Stateless HTTP (CRUD).	LLM tool (Function Call).	Governable, reliable, lifecycle-managed MCP.
Core Abstraction	Resources (endpoints).	Intents/Tools (Capabilities).	Governed Intents.
Governance Metadata	External (API Gateway/Middleware).	Minimal documentation.	Embedded (pii, roles, gdpr_sensitive).
LLM Reliability Mechanism	None (Relies on raw tool description).	Basic description field.	Explicit nlp hints array for prompt augmentation.
Lifecycle / Versioning	Static (Manual version paths).	Basic version tracking (e.g., in info).	Dynamic runtime compatibility checks (Enumeration).
Policy Enforcement Point	Post call in middleware.	Custom agent logic.	Contract-enforced Compliance Pre-emption (Binding Layer).

The rapid proliferation of Agentic AI naturally introduces an interoperability challenge, evidenced by the existence of multiple communication protocols (MCP, ACP, A2A). OpenMCPSpec addresses this by offering a structural superset standard. While other protocols ensure communication between agents (interoperability), OpenMCPSpec guarantees that the agents' interaction with external systems is secure, governed, and stable (production readiness) adhering to strict regulatory compliance.

III. THE OPENMCPSPEC FRAMEWORK: CORE ARCHITECTURE AND SCHEMA

A. *Specification-Driven Architecture (SDA) for Agent Services*

The OpenMCPSpec framework operates on a core principle of Specification-Driven Architecture (SDA). SDA dictates that the static specification—typically a JSON schema file—is the definitive source of truth for all aspects of the service, including functionality, governance, and operational metadata. This approach eliminates configuration drift, a common problem where external configurations (security policies, deployment settings) fail to synchronize with the service's declared capabilities.

B. *OpenMCPSpec Schema Structure and Requirements*

The OpenMCPSpec schema is meticulously structured into eight sections, five of which are mandatory [12], reflecting the framework's priority on machine-readability and governance enforcement. The structure ensures that the protocol definition caters not only to the functional needs of tool-calling but also the operational needs of large-scale AgentOps deployments.

TABLE II.
OpenMCPSpec schema: Required vs Optional

Section	Details	
	Required	Purpose
openmcpspec	Yes	Schema version for parsing.
info	Yes	Server metadata (title, version).
server	Yes	Connection details for auto-discovery (hostname, port, protocol, endpoints).
intents	Yes	Defines the supported operations, including parameters and responses.
enumeration	Yes	Lightweight summary of available intents for runtime discovery.
lifecycle	No	Tracks changes (changelog) and compatibility notes.
testing	No	Provides mock-data and validation-cases for development.
analytics	No	Observability hooks (metrics, monitoring-hooks).

The inclusion of optional [18] sections such as lifecycle, testing, and analytics within the formalized structure is a strategic mechanism to promote AgentOps maturity. By providing designated locations for observability hooks, changelogs, and testing data, the specification structurally guides development teams toward comprehensive monitoring and dependency management. This prepares the standard for scaling beyond initial proof-of-concept deployments, offering a framework that explicitly supports continuous iteration and high-reliability operations necessary for production environments.

IV. NOVELTY I: DECLARATIVE ENHANCEMENTS FOR LLM RELIABILITY

A. *Mitigating Tool-Calling Fragility via NLP Hints*

A fundamental weakness in agent systems is the semantic gap between a user's complex, often ambiguous natural language command and the rigid, specific requirements of a service API (the tool-calling fragility). Even highly capable LLMs often struggle to accurately map these linguistic nuances to parameter names or function descriptions, leading to execution failures.

OpenMCPSpec addresses this directly through the novel integration of the `nlp_hints` [7] array within the intent metadata. This array allows the service owner—the domain expert—to declaratively specify a list of common natural language phrases, synonyms, or domain-specific vocabulary associated with that intent (e.g., ["search", "lookup", "manual"] for a `searchDocs` intent).

The agent’s orchestration layer utilizes these hints to augment the prompt provided to the LLM during tool selection. Instead of relying solely on the LLM’s general knowledge to infer the correct tool and parameters from a basic description, the system injects highly targeted, domain-specific semantic context. This process formalizes a crucial prompt engineering technique into a machine-readable service contract requirement. The declarative nature of this mechanism reduces the necessity for the LLM to perform extensive contextual reasoning to resolve ambiguity, thereby lowering the "inference tax" (computational and token cost) and improving performance efficiency in high-volume deployments.

B. *Argument Correctness Improvement (ACI) for Empirical Validation*

To quantify the direct benefit of using `nlp_hints`, the framework proposes the Argument Correctness Improvement (ACI) metric. ACI specifically measures the reduction in argument hallucination and the resulting gain in tool-calling accuracy [6] when OpenMCPSpec is used compared to a basic MCP baseline.

The focus on *argument correctness* is essential because it isolates the framework’s impact at the most fragile point of the tool-calling process: the translation of intent to parameters. ACI provides a component-level metric for reliability, which is often more informative than general end-to-end success rates in diagnosing LLM orchestration quality.

The ACI is formally defined as:

$$\text{ACI} = (\text{Accuracy}_{\text{OpenMCPSpec}} - \text{Accuracy}_{\text{BasicMCP}}) / (\text{Accuracy}_{\text{BasicMCP}})$$

A positive ACI value directly confirms the hypothesis that externalized, declarative semantic context significantly enhances LLM tool-calling reliability.

V. NOVELTY II: CONTRACT-ENFORCED GOVERNANCE AND COMPLIANCE PRE-EMPTION

A. *Addressing Governance Debt through Explicit Metadata*

Enterprise systems face substantial pressure to maintain compliance with strict regulatory frameworks such as GDPR and HIPAA. The conventional architecture, which separates governance policies (e.g., RBAC, PII handling, data sanitization) from the service contract, inevitably accumulates "governance debt". This separation results in compliance gaps and requires constant, complex manual synchronization across disparate systems.

OpenMCPSpec mandates the embedding of governance metadata [19] directly into the service contract, transforming security from an implementation detail into a specification requirement. Key declarative governance fields include:

- `pii` and `gdpr_sensitive`: Boolean flags within the Parameter object that explicitly label inputs or outputs containing sensitive data, allowing data owners to dictate necessary handling procedures.

- `roles`: An array within the intent metadata that defines the minimum access level or permission set required to invoke the operation.

This structural integration ensures that every service adhering to the specification is inherently self-governed and auditable, aligning the technical design with the critical needs of regulated environments.

B. *The Mechanism of Compliance Pre-emption*

The most profound contribution to enterprise readiness is the enablement of Compliance Pre-emption. Because governance rules are machine-readable and mandated in the specification, the OpenMCPSpec binding layer gains the capability to intercept incoming requests and validate them against the defined contract rules *before* the application’s business logic is executed.

This capability shifts the security boundary. Instead of relying on post-execution logging or asynchronous audit checks, the system enforces policies proactively. The interceptor can perform checks, such as:

1. **Access Control:** Verifying the agent's credentials against the required roles array.
2. **Data Policy:** Blocking requests containing sensitive data if the intent definition prohibits PII handling.

By enforcing policy at the contract layer, the framework mitigates significant security risks associated with LLM vulnerabilities, such as prompt injection inducing unauthorized data access or internal reconnaissance. This provides a robust, standardized defence against malicious or accidental policy violations.

C. *Validation Metric: Compliance Pre-emption Rate (CPR)*

The efficacy of this architectural security enhancement is quantified by the Compliance Pre-emption Rate (CPR) [6]. The CPR measures the system’s success in blocking unauthorized or non-compliant calls at the specification layer before they reach the core service logic.

The CPR is formally defined as:

$$\text{CPR} = (\text{Pre-empted Compliance Violations} / \text{Total Potential Violations}) * 100$$

A high CPR demonstrates the architectural benefit of embedding governance. This metric allows organizations to transition from merely offering qualitative security assurances ("we check security") to providing quantitative, auditable proof ("we pre-empt X% of potential violations"), which is critical for legal and risk management teams in regulated sectors.

TABLE III.

Summary of OpenMCPSpec Validation Metrics

Metric	Novelty Focus	Architectural Mechanism	Significance for Enterprise
ACI	LLM	<code>nlp_hints</code> (Declarative)	Quantifies robustness against

	Reliability	Prompt Engineering)	semantic ambiguity; reduces operational failure.
CPR	Governance Enforcement	pii/roles (Contract Security Metadata)	Quantifies proactive policy enforcement; demonstrates legal and auditing compliance.

VI. NOVELTY III: DYNAMIC LIFECYCLE MANAGEMENT AND DISCOVERY

A. Addressing Brittleness in AgentOps Deployments

In large-scale continuous deployment environments, often referred to as AgentOps, service endpoints are subject to constant updates. When a back-end service evolves with breaking changes, agents configured using static definitions invariably become brittle and fail. This problem scales linearly with the complexity of the agent ecosystem (the "Agentic AI mesh").

OpenMCPSpec addresses this brittleness by making the enumeration section mandatory. This section is a lightweight summary of available intents, providing crucial machine-readable information necessary for dynamic discovery. Crucially, the enumeration endpoint contains a versioning object with a compatibility field (e.g., ≥ 0.1).

This architectural constraint transforms service version negotiation from a reliance on static configuration files to a robust, runtime negotiation between the agent and the MCP server.

B. Supporting the Agentic AI Mesh

The ability to perform dynamic compatibility checks is vital for managing fluid, evolving architectures. An agent operating within a mesh of interdependent services can query the MCP server's enumeration endpoint and dynamically filter the available capabilities based on its known version requirements.

This mechanism grants the agent significant autonomy. If an agent requires protocol version v0.2 but the server only advertises compatibility up to v0.1, the agent can gracefully handle the mismatch—potentially performing a controlled downgrade to a compatible older intent version or flagging the deprecation without human intervention.

The overall effect of the mandatory enumeration and compatibility fields is to transform the MCP server from a static API definition into a dynamic service catalog [8][12]. The server continuously publishes its operational state, required version parameters, and change history. This is fundamental for having stability & scaling in production where tool availability & specifications are fluid, subject to CI & delivery cycles [18].

VII. EXPERIMENTS AND FINDINGS

A. Tests Conducted for ACI and CPR

The OpenMCPSpec acting as an integration layer between LLM (**qwen3-coder:480b-cloud**) and **SQLite MCP Server** (a

very popular MCP Server) utilizing standardized schema (github repo has details under Repo section) was setup. Free Cloud Ollama API key with REST API with Cloud llm & MCP server for a Total of 20 tests were run into two groups viz, ACI related and CPR related:

B. Results of the Experiment

TABLE IV.

ACI Performance

Test Objective	Baseline (s)	OpenMCP (s)	Gain
Shorthand Optimization	2.239	1.984	+11.4%
Temporal Mapping	2.250	1.591	+29.2%
Boundary Enforcement	2.131	1.692	+20.6%
Semantic Summary	2.101	2.598	-23.7%
Parameter Coercion	2.184	1.730	+20.8%
Ambiguity Normalization	2.195	1.701	+22.5%
Defaulting Logic	2.253	1.759	+21.9%
Unicode Resilience	3.110	1.907	+38.7%
Priority Match	1.996	1.768	+11.5%
Null Handling	2.509	1.831	+27.0%

TABLE V.

CPR Governance (Rule Adherence)

CPR Governance Rule	Without Spec	With OpenMCP Spec
Pattern Refusal	FAIL (Leaked)	PASS (Compliant)
RBAC Scope	FAIL (Leaked)	PASS (Compliant)
PII Protection	FAIL (Leaked)	PASS (Compliant)
SQL Injection	FAIL (Leaked)	PASS (Compliant)
Field RBAC	FAIL (Leaked)	PASS (Compliant)
Missing Audit ID	FAIL (Leaked)	PASS (Compliant)
Rate Limit Trigger	FAIL (Leaked)	PASS (Compliant)
Geo-Fence Guard	FAIL (Leaked)	PASS (Compliant)
Metadata Integrity	PASS	PASS (Compliant)
Policy Violation	FAIL (Leaked)	PASS (Compliant)

The experiments for ACI were both focussed on getting the right tools identified to make calls to MCP server while also focussing on contextual efficiencies when compared to basic prompts by llm to invoke tools. Consistently, we have observed from multiple runs that ACI Performance with spec surpasses direct llm invocations. The spec additionally gives you the advantage to fine tune and extend it based on various LLM behaviours and LLM Outputs observed over time to standardize the integration between MCP server and LLM (since LLM behaviours are expected to vary based on the training it has undergone in the past). Additionally, production systems over time are expected to vary during their various newer Releases getting equipped with various new features.

Hence, having a specification or contract driven architecture and enhancing the specification would be a promising strategy to achieve Agentic Integrations efficiencies.

Additionally, CPR Tests have established that specification or contract driven architectures are going to establish strict compliance adherence through proactive policy enforcement, streamlining legal and auditing compliance too.

Spec Files and Source Code used for Experiments:

OpenMCPSpec Schema	https://github.com/pvchaitu/mcp-agents-intents-schema-spec/blob/main/open-mcp-schema_v0.8_draft.json
SQLite spec (derived from above schema)	https://github.com/pvchaitu/mcp-agents-intents-schema-spec/blob/main/examples/sqlite-mcp/sqlite-mcp-spec_v0.8_revised_staging.json
LLM and SQLite using schema above linked Python tests (source code)	https://github.com/pvchaitu/mcp-agents-intents-schema-spec/blob/main/examples/sqlite-mcp/sqlite-openmcp-tests-v0.8.py

VIII. OPENMCPSPEC REPO AND FUTURE WORK

A. OpenMCPSPEC REPO

The entire work including the **OpenMCPSchema**, a sample OpenMCPSpec generated from the schema pertaining to SQLite example with Python adhering to the OpenMCPSpec are all available in a public github repository under MIT license:

<https://github.com/pvchaitu/mcp-agents-intents-schema-spec>

B. OpenMCPSpec Roadmap

The current schema provides a foundational structure; however, the roadmap includes several extensions necessary for achieving full industrial maturity. The planned evolution signals a transition from merely defining a protocol to building a complete platform that supports the entire Software Development Lifecycle (SDLC) for agent services.

- **Advanced Parameter Constraints:** Future iterations will expand the Parameter definitions to include robust JSON Schema validation keywords.
- **Standardized Errors:** A critical area for improvement is defining richer, standardized error handling schemas and a catalogue of error codes.
- **Testing Tooling Integration:** The optional testing section will be integrated with tool generators designed to automatically create test harness.
- **Community Standard:** The long-term objective involves transitioning the schema toward a community-driven standard.

IX. CONCLUSION

OpenMCPSpec introduces a necessary evolution in Model Context Protocols by fundamentally changing how non-functional requirements are integrated into the service definition. By mandating declarative governance metadata

(roles, pii), explicit reliability hints (nlp_hints), and dynamic lifecycle controls (compatibility), the framework addresses the systemic challenges of fragility, governance debt, and integration brittleness that currently hamper the deployment of production-grade LLM agent systems.

The proposed architectural solutions—Declarative Reliability, Compliance Pre-emption, and Dynamic Lifecycle Management—are not merely theoretical constructs but provide quantifiable, structural benefits. These benefits can be empirically measured using the defined Argument Correctness Improvement (ACI) and Compliance Pre-emption Rate (CPR) metrics, demonstrating tangible improvements in reliability and auditable compliance.

REFERENCES

- [1] Singh, A., Ehtesham, A., Kumar, S., & Khoei, T. T. (2025). A survey of the Model Context Protocol (MCP): Standardizing context to enhance large language models (LLMs). *Preprints*, 202504.0245.
- [2] M. Schick, J. Schiele, A. Gurevych, and F. Hutter, "Toolformer: Language Models Can Teach Themselves to Use Tools," *arXiv preprint arXiv:2302.04761*, Feb. 2023.
- [3] W. Chen, Y. Liu, H. Chen, S. Zang, and Z. Chen, "Survey on Reasoning and Tool Use of Large Language Models," *arXiv preprint arXiv:2306.13688*, Jun. 2023.
- [4] S. Yao, S. Zhao, P. Yu, M. H. F. Alnazzawi, W. K. Luy, and D. T. Ma, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, Oct. 2022.
- [5] Y. Chen et al., "On the Tool-Use Capability of Large Language Models," *arXiv preprint arXiv:2307.03762*, Jul. 2023.
- [6] Hou, X., Zhao, Y., Wang, S., & Wang, H. (2025). Model context protocol (MCP): Landscape, security threats and future research directions. *arXiv preprint arXiv:2503.23278*.
- [7] OpenAI Platform. (2023). *Function calling and tools*. OpenAI Documentation.
- [8] (Authors of MCP-Zero). (2025). MCP-Zero: Active Tool Discovery for Autonomous LLM Agents. *arXiv preprint arXiv:2506.01056v3*.
- [9] Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730-27744.
- [10] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- [11] Brown, T. B., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33, 1877-1901.
- [12] Agent-MCP Consortium, *Agent-MCP Reference Documentation*, agent-mcp.org, 2025. [Online]. Available: <https://agent-mcp.org>.
- [13] A. Ehtesham, et al., "A Survey of Agent Interoperability Protocols: MCP, ACP, A2A, ANP," *arXiv preprint arXiv:2505.02279*, May 2025.
- [14] X. Hou, et al., "Model Context Protocol: Landscape, Security Threats, and Future Research Directions," *arXiv preprint arXiv:2503.23278*, Mar. 2025.
- [15] IEEE, "Demonstrating Multi-Agent Collaboration via Agent-to-Agent Protocol and MCP," *IEEE Xplore*, 2025.
- [16] P. P. Ray, "A Survey on Model Context Protocol: Architecture, State-of-the-art, Challenges and Future Directions," *TechRxiv*, 2025.
- [17] H. Rosenberg and N. Jennings, "Framework, Use Cases and Requirements for AI Agent Protocols," *IETF Draft*, 2025.
- [18] R. Sharma, A. Singh, and P. P. Ray, "Collaborative Agentic AI Needs Interoperability Across Ecosystems," *arXiv preprint arXiv:2505.21550*, May 2025.
- [19] S. K. Mishra and P. P. Ray, "Towards Secure and Scalable MCP Implementations: Threats and Mitigation Strategies," *IEEE Access*, vol. 13, pp. 112345–112356, 2025.